Task Manager Codebase Understanding – Summary

1. Project Overview

The Task Manager is a **Python-based CLI application** that allows users to create, update, delete, and manage tasks. It uses only the Python standard library and follows a modular design.

---

2. Codebase Structure

The system is divided into key components:

- **CLI Layer (cli.py)** → Handles user commands and input
- **Business Logic (task_manager.py)** → Core task operations
- **Models (models.py)** → Defines Task, Priority, and Status
- **Storage (storage.py)** → Saves and loads tasks using JSON
- **Scoring System (task_priority.py)** → Calculates task importance
- **Parser (task_parser.py)** → Converts natural language into tasks
- **Merge System (task_list_merge.py)** → Syncs local and remote tasks

---

3. Key Technologies

- Python 3.11+
- Standard libraries only:
    - argparse
    - datetime
    - json
    - uuid
    - re
    - os
    - copy

---

4. Main Domain Model

The system revolves around the **Task entity**, which includes:

- Title, description

- Priority (LOW → URGENT)
- Status (TODO → DONE)
- Due dates
- Tags
- Timestamps

Tasks are:

- Scored based on urgency
- Sorted by importance
- Stored in JSON format
- Updated through CLI commands

---

5. Feature Implementation Insight (CSV Export)

- Existing storage uses JSON serialization
- Best place for CSV export: storage.py or new export.py
- Would require:
    - Extracting tasks
    - Formatting as CSV
    - Writing using Python csv module
    - Adding CLI command

---

6. Business Rule Design

New rule:

Tasks overdue by more than 7 days become “abandoned” unless HIGH priority.

Implementation would involve:

- Adding new status: ABANDONED
- Checking overdue duration in TaskManager
- Updating task status automatically

---

7. Key Insights from Codebase Analysis

- The system is more advanced than a basic task manager
- It includes:
    - Intelligent scoring system
    - Natural language parsing
    - Task merging and sync logic
- storage.py is the central persistence layer
- task_priority.py controls task importance logic

---

8. Learning Outcome

This exercise improved understanding of:

- How to explore unknown codebases
- How to identify system architecture
- How business logic is distributed across modules
- How to plan feature implementation before coding

---

9. Final Conclusion

The Task Manager is a **modular, rule-based task system** that combines CLI interaction, intelligent scoring, and structured data storage. AI-assisted prompts helped quickly map the system architecture and identify where new features should be implemented.